Stellar Cyber 6.6.0s is now live! Discover all the updates in the release notes!

Ingesting Logs over TLS

You can configure Modular Sensors to ingest the following logs encrypted via Transport Layer Security (TLS):

- rsyslog
- syslog-ng
- SentinelOne logs
- Palo Alto Cortex

To configure TLS log ingestion:

1. Configure Stellar Cyber to ingest the logs.
2. Configure the log sender to send the logs to the Modular Sensor.

Configure Log Ingestion over TLS in Stellar Cyber

Stellar Cyber provides two options for a Modular Sensor to use TLS to secure log ingestion:

- Use the Stellar Cyber self-signed CA Certificate. Download the certificate from the Edit Sensor Parameters dialog box in Stellar Cyber and install it in the log sender.

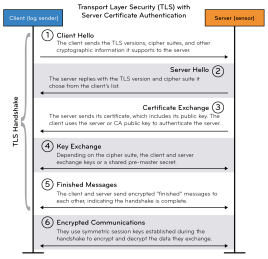
  or

- Use a certificate purchased from a trusted commercial vendor. Upload the certificate and private key to the Stellar Cyber Platform on the System | Saved Objects | Certificates page and apply the certificate to a Modular Sensor on the System | DATA SOURCE MANAGEMENT | Sensors | Sensors page.

  This technique is useful in situations where a vendor's deployment does not trust the self-signed certificate provided by Stellar Cyber (for example, Palo Alto Cortex).

TLS Handshake and Server Certificate Authentication

A TLS handshake is a series of exchanges between a client and server to establish a secure connection across an unsecured network (Internet). In terms of Stellar Cyber, the client is a log sender and the server is a Stellar Cyber Modular Sensor. The exchanges that occur during the handshake are presented in the following illustration.

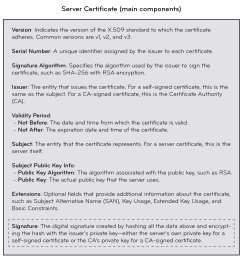


The server certificate that the Modular Sensor uses can either be signed by the server itself or by a Certificate Authority (CA). Here's what's involved with the creation and delivery of a self-signed server certificate:

1. The server generates a public-private key pair.
2. The server generates a server certificate containing its public key and other information, such as its version, serial number, and signature algorithm.
3. The server generates a hash, or *message digest*, of the certificate contents and encrypts the hash with its private key to create a digital signature.
4. The server appends the digital signature to the server certificate, making it a self-signed certificate.
5. The self-signed server certificate, which includes its public key, must be delivered to the client out of band—for example, on a USB drive or secure email—and added to the client's certificate store.

The creation and delivery of a CA-signed server certificate involves the following:

1. The server generates a public-private key pair.
2. The server creates a Certificate Signing Request (CSR), containing its public key and other information, and sends it to a CA.
3. The CA generates a hash and encrypts it with its private key, creating a digital signature.
4. The CA appends the digital signature to the server certificate, making it a CA-signed certificate.
5. The CA sends the certificate back to the server. The CA-signed server certificate includes the server's public key, the CA-encrypted digital signature, and other information.



The verification process of the server certificate and, by extension, the server itself proceeds like this:

1. The client extracts the digital signature and public key from the self-signed certificate or the public key from the CA certificate, which presumably it already has in its certificate store.

2. The client uses the public key to decrypt the digital signature.
3. The client generates a hash of the server certificate and compares it with the decrypted hash. If they match, the certificate is verified as authentic and unmodified.
4. The client uses the server's public key to encrypt a pre-master secret, which it sends to the server.
5. The client and server use the pre-master secret to generate symmetric session keys to encrypt and decrypt data transmitted in their session.

Configure Log Ingestion over TLS Using a Self-Signed Certificate

To configure TLS log ingestion in Stellar Cyber using the self-signed certificate:

1. Select **System | DATA SOURCE MANAGEMENT | Sensors | Sensors**.

   The **Sensor List** appears.

2. Select for the Modular Sensor that you want to ingest TLS logs.

   The **Edit Sensor Parameters** window appears.

3. Toggle on **Syslog TLS Enabled**.
4. Select **CA Certificate** to download the self-signed public certificate for the Modular Sensor.
5. Select **Submit**.
6. Open the appropriate TCP ports on your firewall: 5140 for rsyslog or syslog-ng, 443 for Cortex, or 5175 for SentinelOne.
7. Install the self-signed public certificate from the Stellar Cyber Modular Sensor on the log sender.

> Some log senders (for example, ESET) include an option to **Validate CA root certificates of TLS connections** when you upload a certificate. Because the self-signed certificate from Stellar Cyber is not a true public certificate, you must disable any options that validate CA root certificates for TLS log ingestion using the self-signed certificate to work.

Configuring Log Ingestion over TLS Using a Commercial Certificate

To configure TLS log ingestion in Stellar Cyber using a commercially purchased certificate:

1. Purchase one of the trusted certificates for your log source.

   For example, here is the list of trusted certificates for syslog and HTTPS forwarding from Palo Alto Networks' Cortex Data Lake. You can also refer to **Generating Server Certificates** for information on generating the Certificate Signing Request (CSR) file you will need to obtain a certificate for your sensor's domain from your service provider.

2. Log in to Stellar Cyber and navigate to the **System | Saved Objects | Certificates** page.
3. Upload the certificate to Stellar Cyber by selecting **Upload**, choosing the **Server Certificate** option, and using the **Choose File** option to select the certificate and private

key you want to use for TLS log ingestion. Refer to these instructions for details on uploading a certificate to Stellar Cyber.

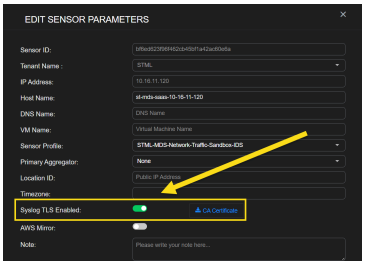
4. Navigate to the **System | DATA SOURCE MANAGEMENT | Sensors | Sensors** page.

   The **Sensor List** appears.

5. Select for the sensor you want to ingest TLS logs.

   The **Edit Sensor Parameters** window appears.

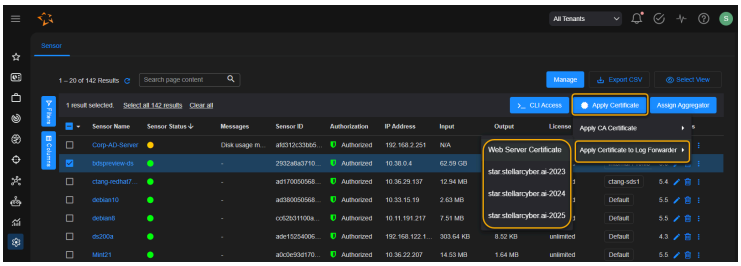
6. Toggle on the **Syslog TLS Enabled** option, as illustrated below.



7. Select the sensor to which you want to apply the certificate by selecting its box in the System | DATA SOURCE MANAGEMENT | Sensors | Sensors list.

8. Apply the certificate to a Modular Sensor using the **Apply Certificate | Apply Certificate to Log Forwarder** button in the System | DATA SOURCE MANAGEMENT | Sensors | Sensors list.

   The drop-down list shows all uploaded server certificates that have appear on the **Certificates** page.



   The user interface does not let you apply a certificate to a log forwarder on a Server Sensor.

   After applying the certificate, the selected sensor uses the selected certificate as the TLS certificate for log ingestion on all parser ports, including those configured in the System | DATA SOURCE MANAGEMENT | Parsers | Log Sources page.

9. Open the appropriate TCP ports on your firewall.

   For Cortex Data Lake, this is typically 443.

10. Configure your log source to send data to the Modular Sensor over the encrypted TLS connection.

Disabling the **Syslog TLS Enabled** option for a Modular Sensor that is using a server certificate removes the association between the sensor and the certificate. If you then re-enable **Syslog TLS Enabled**, the sensor reverts to using the self-signed certificate instead of using the server certificate that was previously configured. If this happens, you can just

reapply the server certificate using the the **Apply Certificate | Apply Certificate to Log Forwarder** button. 5.3.0

Refer to **Removing a Certificate from a Sensor** for information on removing a log forwarder certificate from a sensor.

Configuring rsyslog to Send Logs to a Modular Sensor

To configure rsyslog to send logs to a Modular Sensor:

1. Install rsyslog.
2. Install rsyslog-gnutls.
3. Edit the `/etc/rsyslog.conf` file to add the following configurations (use your sensor IP address for the target):

```
global(DefaultNetstreamDriverCAFile="/etc/pki/tls/private/cacert
.pem")

action(type="omfwd" target="1.1.1.1" protocol="tcp" port="5140"
StreamDriver="gtls" StreamDriverMode="1"
StreamDriverAuthMode="anon")
```

4. Restart the rsyslog service.

Use our example as a guideline, as you might be using a different software version.

The logs are now sent to the Modular Sensor and ingested.

Configuring syslog-ng to Send Logs to a Modular Sensor

To configure syslog-ng to send logs to a Modular Sensor:

1. Install the syslog-ng client.
2. Edit the `/etc/syslog-ng/syslog-ng.conf` file to add the following configurations (use your sensor IP address for the network and syslog):

```
destination demo_tls_destination {
   network("1.1.1.1" port(5140)
     transport("tls")
       tls( ca-dir("etc/syslog-ng/cacert.pem"))
     );
   };
 destination demo_tls_syslog_destination {
   syslog("1.1.1.1" port(5140)
     transport("tls")
       tls(ca-dir("/etc/syslog-ng/cacert.pem"))
     );
   };
```

```
log { source(s_sys); destination(demo_tls_syslog_destination);};
log { source(s_sys); destination(demo_tls_destination);};
```

3. Restart the syslog-ng service.

Use our example as a guideline, as you might be using a different software version.

The logs are now sent to the Modular Sensor and ingested.

Forwarding Logs from a Windows Server Sensor

You can also configure a Windows Server Sensor to act as a syslog forwarder, using its built-in Filebeat service to receive syslog records from a specified source and forward them with TLS encryption to a Modular Sensor for parsing, filtering, and eventual shipment to the Stellar Cyber Platform. This feature is useful in situations where it might not make sense to deploy a dedicated Modular Sensor for syslog ingestion. Refer to **Configuring Filebeat Settings** for details.

© 2026 Stellar Cyber . All rights reserved.